RTA Fines & Salik Engine

Integration proposal for the Pilot GPS platform

Prepared by TSC (Sky Electronics) — Pilot partner #207, UAE

October 2026 · Version 1.1

Goal: make RTA fines and Salik tolls a native, first-class part of Pilot — on the vehicle card and in reports, for every UAE fleet.

1. Summary

TSC operates a live fines-and-tolls data engine for UAE vehicle fleets: RTA traffic fines (Dubai letter-code plates via the comgw.rta.ae gateway, and UTS-coded plates for Sharjah, Abu Dhabi, Ajman, UAQ, RAK and Fujairah) plus Salik toll trips from two Salik portal accounts. It is running today for the RMA rental fleet on your platform, delivered as a Pilot Store extension.

This document is the handover for native integration: it gives your development team the exact data sources, the already-running feed infrastructure, the API contracts, and the concrete places in the Pilot application where the data should surface. Our side keeps producing the data; your side adds a small backend integration and the UI surfaces the values for every UAE fleet — not just one extension.

2. What runs today (our side)

Two automated pipelines refresh the data and expose it as JSON:

- **RTA fines pipeline** — a connector talks to comgw.rta.ae (AES-128-CBC encrypted envelopes) and to the UTS plate-code lookups for the other emirates. Results are cached per plate and served by our API.
- **Salik pipeline** — an hourly cron job (UAE-side machine, because the Salik portal blocks foreign IPs) logs into the two Salik accounts, pulls 30 days of trip history per plate, and pushes a merged JSON snapshot to our server.
- **API layer** — FastAPI service at **https://pilot-ai.zamtek.ae** serves the combined data, the AI support agent and email reports.

Live API endpoints

Endpoint	Method	Purpose
/salik	GET	30-day Salik tolls per plate (merged multi-account snapshot)
/rta-fines	POST {plates: []}	RTA payable fines per plate (cached 1h)
/ask	POST	AI support agent with fleet context (optional)
/report	POST	Email a generated report (optional)

These endpoints exist now and will keep running. Integration does not require rewriting them — Pilot can consume them directly, or we push the data to a Pilot-side endpoint (Section 4).

3. Data formats

3.1 Salik snapshot (/salik)

```
{ "matched": 82, "notFound": 67, "totalTolls": 11405.1, "days": 30,
  "data": { "AA35668 Duster": { "salikPlate": "35668:AA",
              "tripCount": 162, "totalTolls": 716.1,
              "tolls": [ {"date": "2026-10-01 08:14", "amount": 4.0,
                          "gate": "Al Barsha"}, ... ] } } }
```

Keys are the vehicle names exactly as they appear in Pilot (plate prefix + model). Matching to a Pilot vehicle is therefore a string match on the vehicle name — no extra mapping needed for the Salik side.

3.2 RTA fines (/rta-fines)

```
POST {"plates": ["DXB K 62968", "SHJ 3 18894"]}
-> {"DXB K 62968": {"fines": [ {"ticketDate": "2026-09-12",
        "description": "Exceeding speed limit", "amount": 600, ... }],
        "totalAmount": 600, "totalBlackPoints": 0, "cached": true},
    "SHJ 3 18894": { ... }}
```

Plate decoding (emirate → UTS code, Dubai letter codes) happens on our side. Pilot only needs to send the vehicle plate string it already has.

4. Integration path on the Pilot side

Recommended path — backend integration, no browser extension, works for all UAE clients:

4.1 Data ingest (backend)

- **Option A (fastest):** a Pilot backend job calls /salik and /rta-fines hourly per partner account, stores per-vehicle values in a new table (vehicle_id, rta_fines_count, rta_fines_amount, salik_30d_trips, salik_30d_amount, checked_at). One endpoint change: accept an optional per-account token we provide.
- **Option B (push):** Pilot adds POST /backend/ax/external_data.php (auth: partner JWT already used by your admin API) accepting the same key/value pairs per agent_id; our cron posts hourly. Payload format in Section 5.

4.2 UI surfaces (frontend)

Surface	File / handler in current build	Change
Vehicle card popup on map	MapContainer.getMarkerContent2() (adds popup HTML rows)	Append 2 rows: RTA fines AED (unpaid), unpaid count
Vehicle editor / card	Pilot.view.online.VehicleEditor	Add a 'Fines & Tolls' fieldset reading the stored
Reports	ax/reports.php / report templates	Two new columns: RTA fines AED, Salik 30d A
Store extension (until native)	pilotstore list.php (app id 165)	Keep published as the interim UI

We verified the popup mechanism against the current app bundle: marker data (agentid in `id`, `veh_id`) is stored per marker via setStorage("data"), and getMarkerContent2() builds the popup HTML the user sees. Adding the two rows there is a 1–2 day frontend change; the same values can feed the vehicle editor and reports without further API work.

5. Push payload (Option B contract)

```
POST https://pilot-gps.com/backend/ax/external_data.php
Authorization:
{ "updates": [
    { "agent_id": 123456,
      "values": { "rta_fines_count": 3, "rta_fines_unpaid": 2,
                  "rta_fines_amount_aed": 1380,
                  "salik_30d_trips": 162, "salik_30d_amount_aed": 716.1,
                  "checked_at": "2026-10-28T13:02:00Z" } }, ... ],
  "account_id": 19336 }
```

6. Estimated effort

Work item	Side	Effort
Consume /salik + /rta-fines hourly, store per vehicle	Pilot backend	2–3 dev days
Popup rows on vehicle card (getMarkerContent2)	Pilot frontend	1–2 dev days
Vehicle editor fieldset + report columns	Pilot frontend/backend	2–3 dev days
Our side: keep pipelines running, push mode (if Option B)	TSC	0 (already running)

7. What we need from Pilot

- Confirm Option A or B; we open a test account if needed.
- For Option A: an API token per partner account for our endpoints.
- For Option B: the endpoint contract above (auth via existing partner JWT).
- Keep the Store extension published until the native path is live.

8. Contact

TSC / Sky Electronics — Pilot partner #207, UAE.
Contact: muhammed.charyyev@gmail.com
Data API: https://pilot-ai.zamtek.ae (health, schemas and a live sample available on request).